ADR-002: Que el módulo de Inventario controle el estado de sus equipos

Decisión de arquitectura de software

Estado: Implementada en el código. Queda pendiente ejecutar la migración V3 en la base de datos MySQL para retirar los triggers.

1. Problema que resuelve (semana 3)

En la semana 3 identificamos que los módulos de Préstamos, Inventario y Usuarios tenían dependencias entre sí. Esto significa que un cambio en uno de los módulos podía afectar el funcionamiento de los demás.

Durante la revisión de la semana 8 encontramos un caso claro: el estado de un equipo, que es un dato central para Inventario, no era actualizado directamente por ese módulo. En su lugar, dos triggers asociados a la tabla de préstamos se encargaban de hacerlo:

- trg_prestamo_insert cambiaba el estado del equipo a “prestado”.
- trg_prestamo_devolucion lo cambiaba nuevamente a “disponible”.

Además, la validación para comprobar que un equipo estuviera disponible se encontraba dentro del procedimiento sp_registrar_prestamo, que pertenece al módulo de Préstamos. Por eso, una regla propia de Inventario estaba repartida entre componentes diferentes.

2. Cambio propuesto

La propuesta consiste en que Inventario sea responsable de controlar el estado de sus equipos. Para conseguirlo, se plantean los siguientes cambios:

1. Crear EstadoObjeto, una clase que concentra los cuatro estados de un equipo y define qué transiciones están permitidas:

- Disponible puede pasar a prestado, mantenimiento o baja.
- Prestado solo puede volver a disponible.
- Mantenimiento puede pasar a disponible o baja.
- Baja es un estado final: no se permite salir de él.

2. Crear InventarioServicio como único componente autorizado para cambiar el estado de un equipo. Antes de aplicar cada cambio, debe comprobar con EstadoObjeto que la transición sea válida.
3. Cuando el módulo de Préstamos necesite marcar un equipo como prestado o disponible, deberá solicitar esa operación a Inventario en vez de modificar el estado directamente.
4. Retirar los dos triggers de la base de datos mediante la migración V3.

3. Código afectado

Antes	Después
Trigger trg_prestamo_insert en la tabla prestamo.	Se retira y su función queda a cargo de InventarioServicio::marcarComoPrestado().
Trigger trg_prestamo_devolucion en la tabla prestamo.	Se retira y su función queda a cargo de InventarioServicio::liberarTrasDevolucion().
La comprobación de disponibilidad estaba dentro de sp_registrar_prestamo.	La regla se centraliza en src/Inventario/Dominio/EstadoObjeto.php.
El estado “disponible” estaba escrito directamente en objetos_disponibles.php.	El nombre del estado se obtiene de EstadoObjeto.
La base de datos ejecutaba los triggers automáticamente.	La migración resources/database/migraciones/V3__refactor_inventario.sql los retira. Si ocurre un problema, V3__refactor_inventario.rollback.sql permite volver a crearlos.

Antes de retirar los triggers, revisamos qué partes del sistema dependían de ellos. Se encontró que solo se activaban con los procedimientos de préstamo y devolución, utilizados por registrar_prestamo.php y devolver.php. Esos archivos ya utilizan el nuevo servicio.

4. Atributo de calidad que mejora (semana 8)

La mejora principal es la mantenibilidad. En la semana 8 identificamos que agregar un estado nuevo, como “reservado”, obligaría a modificar al menos tres lugares diferentes, dos de ellos dentro del módulo de Préstamos.

Con el cambio propuesto, la definición de los estados y sus transiciones queda centralizada en EstadoObjeto. Para incorporar un estado nuevo, se ajustaría esa clase y la tabla de equipos, sin tener que modificar archivos del módulo de Préstamos.

Otras opciones que descartamos

- Conservar los triggers y agregar también el servicio: se descartó porque habría dos componentes diferentes capaces de modificar el mismo dato, lo que duplicaría la responsabilidad.
- Mover los triggers a la tabla de equipos: se descartó porque la regla de negocio seguiría oculta en la base de datos y sería más difícil probarla.

Ventajas y costos

- Ventaja: las reglas de cambio de estado quedan centralizadas, son más fáciles de entender y pueden probarse de forma específica.

- Costo: si alguien registra un préstamo directamente desde MySQL Workbench, el estado del equipo ya no se actualizará automáticamente mediante los triggers. Por eso, las operaciones deben realizarse desde la página de la aplicación.

Cómo lo comprobamos

- Las pruebas automáticas deben confirmar que el equipo cambia a “prestado” al registrar el préstamo y vuelve a “disponible” al devolverlo, sin depender de triggers.
- También deben comprobar que se rechace una transición no permitida, como pasar de “baja” a “disponible”.
- Después de ejecutar la migración, verificar en MySQL Workbench que SHOW TRIGGERS FROM prestamos_tec; no devuelva triggers.